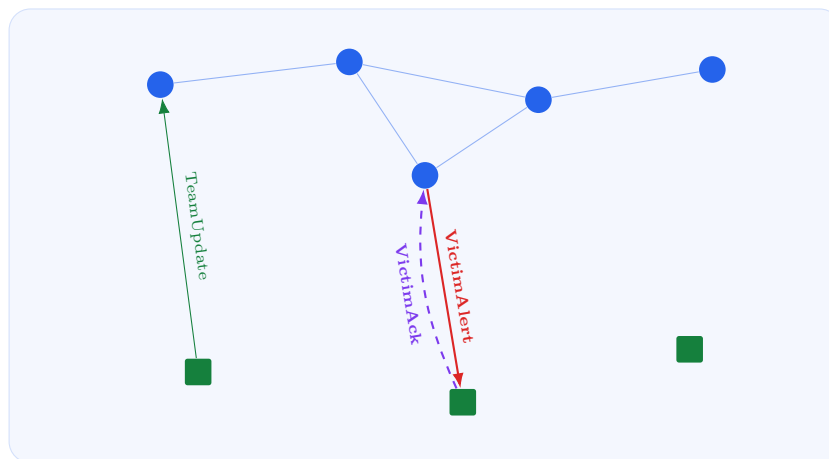


ECHOSAR-Net

Guia Ilustrado dos Logs de Simulação



Como ler, entender e explicar cada linha de log da simulação

Simulação OMNeT++ 6.2.0 + INET 4.5.4 — Dissertação PPGCAP/UDESC

Sumário

1	Por que este guia existe	2
1.1	Como gerar um log legível	2
2	Quem fala com quem: a arquitetura de mensagens	3
2.1	Diagrama de sequência do fluxo completo	3
3	Anatomia de cada mensagem	4
3.1	[DRONE] TeamUpdate — beacon da equipe	4
3.2	[DRONE] DroneStatus — resposta do drone	4
3.3	[DRONE] VictimAlert — o alerta	4
3.4	[TEAM] VictimAck — a confirmação	4
4	Por baixo do capô: rádio, MAC e roteamento AODV	5
4.1	Camada física (PHY) — rádio 802.11	5
4.2	Camada de enlace (MAC) — handshake 802.11	5
4.3	Roteamento AODV — descoberta de rota	5
5	Timers e parâmetros de controle	7
5.1	Timers internos do drone (SimpleDroneApp)	7
6	Log real anotado — uma simulação inteira em miniatura	8
7	Métricas gravadas ao final (finish())	9
7.1	Do lado do drone	9
7.2	Do lado da equipe	9
8	Glossário rápido	10

1 Por que este guia existe

Este documento explica, item por item, o que aparece nos logs quando a simulação ECHOSAR-Net roda. O objetivo é permitir apresentar e defender cada linha de log para a banca/orientação: o que ela significa, de onde vem no código, e por que aquele valor apareceu ali.

Ideia central da simulação

Drones sobrevoam uma área simulando busca e resgate (SAR). Não há vítimas físicas nem sensores reais de detecção — os drones geram **alertas sintéticos** em intervalos aleatórios para gerar tráfego de rede representativo. O que se mede é a **qualidade da comunicação** entre drones e equipes de resgate em terra: o alerta chega? Em quanto tempo? Quantas retransmissões precisou?

1.1 Como gerar um log legível

Por padrão, a simulação roda em modo “express” (rápido, sem imprimir eventos) para não desperdiçar tempo de CPU. Para ver as mensagens de aplicação ([**DRONE**], [**TEAM**]), é preciso pedir o nível **INFO** explicitamente:

```
./run.sh --info -c SmokeTest_Beacons
```

O que a flag `-info` faz de verdade

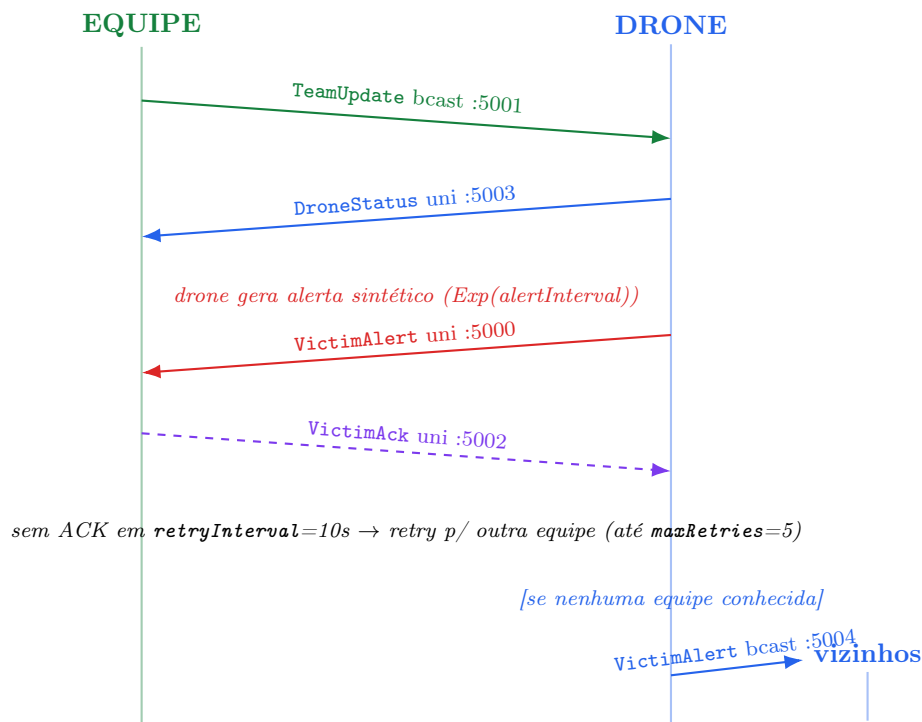
O script `run.sh` passa duas flags ao motor de simulação: `-cmdenv-log-level` (qual severidade mostrar) e `-cmdenv-express-mode` (se os eventos são impressos ou não). **As duas precisam estar certas** — só pedir `log-level=INFO` não basta se o `express-mode` estiver ligado, porque ele suprime a impressão de qualquer evento independente do nível. `-info` cuida das duas coisas.

2 Quem fala com quem: a arquitetura de mensagens

Cinco tipos de mensagem UDP circulam na rede, cada uma numa porta fixa (`src/app/ports.h`):

Porta	Mensagem	Sentido	Quando é enviada
5000	VictimAlert	drone → equipe	Drone gera alerta sintético e escolhe a equipe mais próxima conhecida
5001	TeamUpdate	equipe → broadcast	Beacon periódico da equipe (posição + IP)
5002	VictimAck	equipe → drone origem	Equipe confirma recebimento do alerta
5003	DroneStatus	drone → equipe	Resposta do drone ao receber um TeamUpdate
5004	VictimAlert	drone → drone (relay)	Drone repassa alerta em broadcast quando não conhece nenhuma equipe

2.1 Diagrama de sequência do fluxo completo



Por que o ACK volta direto ao drone que criou o alerta, não a quem repassou

O campo `originIp` dentro do **VictimAlert** guarda o IP do drone que **originou** o alerta. Mesmo que a mensagem tenha passado por vários relays até chegar à equipe, o **VictimAck** é endereçado direto a esse IP de origem via roteamento AODV multi-hop — não ao último drone que fez relay.

3 Anatomia de cada mensagem

3.1 [DRONE] TeamUpdate — beacon da equipe

```
[INFO] [TEAM team0] TeamUpdate broadcast (ip=10.0.0.2)
[INFO] [DRONE drone[0]] tabela: team0 ip=10.0.0.2
```

Campos do TeamUpdateChunk — 128 bytes

teamId (nome da equipe) · **ipAddress** (IP dela na sub-rede 10.0.0.x/24) · **posX/posY** (posição cartesiana no playground, em metros).

O que acontece: a equipe transmite isso em broadcast a cada **sendInterval** segundos (com um jitter aleatório inicial para não colidir com outras equipes no rádio). Todo drone que recebe atualiza sua **teamTable** — é assim que o drone *descobre* quais equipes existem e onde estão, sem nenhuma configuração estática.

3.2 [DRONE] DroneStatus — resposta do drone

```
[INFO] [TEAM team0] DroneStatus de drone[0] pos=(2500,2500,100) RTT=0.004653659192s
```

Campos do DroneStatusChunk — 128 bytes

droneId · **posX/posY/posZ** (posição 3D, incluindo altitude) · **sentAt** (timestamp de envio, usado para calcular o RTT mostrado no log).

O que acontece: é a resposta automática do drone a todo **TeamUpdate** recebido — funciona como um “ping” de posição. A equipe usa isso para saber que drones estão por perto e ativos (não é usado para nenhuma métrica de entrega, é diagnóstico de conectividade).

3.3 [DRONE] VictimAlert — o alerta

```
[INFO] [DRONE drone[0]] alerta sintético gerado → drone[0]_1
[INFO] [DRONE drone[0]] VictimAlert drone[0]_1 → team0 (10.0.0.2)
[INFO] [TEAM team0] *** ALERTA de drone[0] msgId=drone[0]_1 vitima em (2500,2500) delay=0.000134368468s
```

Campos do VictimAlertChunk — 256 bytes

droneId · **msgId** (identificador único **droneId_N**, usado para deduplicação) · **originIp** (IP de quem originou, para o ACK voltar certo) · **posX/posY** (posição da “vítima”) · **sentAt**.

O que acontece: a cada intervalo exponencial (**alertInterval**), o drone gera um alerta sintético e escolhe a equipe conhecida **mais próxima** da sua **teamTable** para enviar via unicast. Se não conhece nenhuma equipe, faz relay em broadcast (porta 5004) para drones vizinhos tentarem entregar.

3.4 [TEAM] VictimAck — a confirmação

```
[INFO] [TEAM team0] VictimAck drone[0]_1 → drone origem 10.0.0.1
[INFO] [DRONE drone[0]] VictimAck recebido para drone[0]_1 de team0
```

Campos do VictimAckChunk — 64 bytes

msgId (referencia o alerta confirmado) · **teamId** (quem confirmou) · **sentAt**.

O que acontece: ao receber e processar um **VictimAlert**, a equipe manda esse ACK de volta ao **originIp**. O drone remove o alerta de **pendingAlerts** e para de retransmiti-lo. Se o mesmo **msgId** chegar de novo (ACK perdido no caminho), a equipe reenvia o ACK sem contar de novo nas métricas — evita retransmitir o alerta inteiro por perda só do ACK.

4 Por baixo do capô: rádio, MAC e roteamento AODV

Tudo que foi mostrado até aqui ([DRONE]/[TEAM]) é log de **aplicação**. Mas cada uma dessas mensagens desce por várias camadas de rede simuladas antes de virar sinal de rádio — e o **-info** imprime o que acontece em *todas* elas. Esta seção explica as linhas mais frequentes desses níveis mais baixos.

4.1 Camada física (PHY) — rádio 802.11

```
[INFO] Radio mode changed from TRANSMITTER to RECEIVER.
[INFO] Changing radio reception state from UNDEFINED to IDLE.
[INFO] Changing radio transmission state from IDLE to UNDEFINED.
[INFO] bpsk snr=6610.06 ber=0
[INFO] 16-Qam snr=6610.06 ber=6.97908e-290
[INFO] Reception ended: successfully for (...)WlanAck (34 us 27 B) ...
      power = 661.006 pW, centerFrequency = 2.412 GHz, bandwidth = 20 MHz
[INFO] Sending up (inet::Packet)WlanAck (14 B)
[INFO] Changing radio reception state from RECEIVING to IDLE.
```

O que cada linha quer dizer

Radio mode / reception state / transmission state: máquina de estados do rádio (IDLE, RECEIVING, TRANSMITTER...) — um rádio half-duplex só pode estar em um estado por vez.
bpsk/16-Qam snr=... ber=...: para *cada* modulação suportada, o simulador calcula a **SNR** (relação sinal-ruído, linear) e a **BER** (taxa de erro de bit) que aquele link teria se usasse essa modulação — serve para decidir se o quadro é decodificado com sucesso.
power = ... pW: potência recebida do sinal, em picowatts — compare com a sensibilidade do receptor (-85 dBm no cenário) para saber se o quadro sequer é detectável.

4.2 Camada de enlace (MAC) — handshake 802.11

```
[INFO] Received frame from PHY: (inet::Packet)WlanAck (14 B)
[INFO] Processing lower frame: WlanAck
[INFO] Finishing last frame sequence step: history = ({ } LAST-FRAME ACK)
[INFO] Processing received frame WlanAck as originator in frame sequence.
[INFO] Frame sequence finished.
[INFO] Channel released.
```

Por que todo pacote de aplicação vira um WlanAck

O IEEE 802.11 exige confirmação em nível de enlace para **todo** quadro unicast: quem envia um DATA espera um ACK do MAC antes de liberar o canal (**Channel released**). Isso é diferente do **VictimAck** da aplicação — este ACK é interno ao rádio, acontece em microssegundos, e existe até para um simples **TeamUpdate**. Se o ACK 802.11 não chegar, o MAC retransmite no nível de enlace, *antes* de qualquer retry da aplicação entrar em ação.

4.3 Roteamento AODV — descoberta de rota

Quando um drone precisa mandar algo para um IP com o qual ainda não tem rota ativa, o AODV descobre a rota sob demanda. Exemplo real de 10.0.0.1 (drone) descobrindo rota para 10.0.0.2 (equipe) para poder enviar um **DroneStatus**:

```
[INFO] Finding route for source <unspec> with destination 10.0.0.2
[INFO] Missing route for destination 10.0.0.2
[INFO] Starting route discovery with originator 10.0.0.1 and destination 10.0.0.2
[INFO] Sending a Route Request with target 10.0.0.2 and TTL= 2
...
[INFO] AODV Route Request arrived with source addr: 10.0.0.1 originator addr: 10.0.0.1 destination addr:
      10.0.0.2
[INFO] I am the destination node for which the route was requested
[INFO] Sending Route Reply to 10.0.0.1
```

```
...
[INFO] AODV Route Reply arrived with source addr: 10.0.0.2 originator addr: 10.0.0.1 destination addr:
      10.0.0.2
[INFO] The Route Reply has arrived for our Route Request to node 10.0.0.2
[INFO] Routing (inet::Packet)DroneStatus (156 B) with destination = 10.0.0.2,
      output interface = wlan0, next hop address = 10.0.0.2
```

Leitura linha a linha

1. **Missing route** — o drone não tem entrada na tabela de roteamento para 10.0.0.2, então dispara descoberta.
2. **RREQ** (*Route Request*) sai em broadcast com TTL=2 — alcança até 2 saltos.
3. Quem recebe o RREQ e **é** o destino responde direto com **RREP** (*Route Reply*); quem não é, encaminharia o RREQ adiante (aqui não precisou, são vizinhos diretos).
4. Ao receber o RREP, o drone grava a rota (**add route**) e agora pode **rotear** o **DroneStatus** de verdade, com **next hop** definido.

Isso só acontece **uma vez** por par origem/destino (enquanto a rota estiver ativa — **activeRouteTimeout=5s**). Depois disso, os pacotes seguintes só aparecem como **Active route found**, sem todo o handshake RREQ/RREP de novo.

5 Timers e parâmetros de controle

Parâmetro	Onde	Papel
<code>alertInterval</code>	drone (.ned)	Média da distribuição exponencial entre alertas sintéticos gerados
<code>teamTimeout</code>	drone (.ned)	Se uma equipe some da <code>teamTable</code> por esse tempo sem novo <code>TeamUpdate</code> , é removida
<code>retryInterval</code>	drone (.ned)	Intervalo entre tentativas de reenvio de um alerta sem ACK
<code>maxRetries</code>	drone (.ned)	Máximo de retransmissões antes do alerta ser descartado (<code>alertsExpired</code>)
<code>sendInterval</code>	equipe (.ned)	Intervalo entre beacons <code>TeamUpdate</code>
<code>beaconJitter</code>	equipe (.ned)	Atraso aleatório do primeiro beacon — evita colisão MAC entre equipes

Por que o `beaconJitter` é crítico

Se todas as equipes transmitirem o primeiro beacon exatamente no mesmo instante, o rádio 802.11 sofre colisões simultâneas e **nenhum** drone recebe `TeamUpdate` — a `teamTable` fica vazia e o PDR despenca para perto de zero. O jitter espalha as transmissões iniciais no tempo. O código rejeita (`cRuntimeError`) se `beaconJitter=0` exatamente por esse motivo já ter causado uma regressão real na simulação.

5.1 Timers internos do drone (SimpleDroneApp)

- `alertTimer` — dispara `generateAlert()` a cada `Exp(alertInterval)`
- `timeoutTimer` — dispara `checkTimeouts()` a cada `teamTimeout`, limpando equipes inativas
- `retryTimer` — dispara `retryPending()` a cada `retryInterval`, reenviando alertas sem ACK

6 Log real anotado — uma simulação inteira em miniatura

Trecho real de `SmokeTest_Beacons` (1 drone parado + 2 equipes, $t = 0$ a 15s) rodado com `./run.sh -info -c SmokeTest_Beacons`. Cada bloco abaixo é um ciclo completo do protocolo:

```
[INFO] [TEAM team0] TeamUpdate broadcast (ip=10.0.0.2)
[INFO] [DRONE drone[0]] tabela: team0 ip=10.0.0.2
[INFO] [TEAM team0] DroneStatus de drone[0] pos=(2500,2500,100) RTT=0.004653659192s
[INFO] [TEAM team1] TeamUpdate broadcast (ip=10.0.0.3)
[INFO] [DRONE drone[0]] tabela: team1 ip=10.0.0.3
[INFO] [TEAM team1] DroneStatus de drone[0] pos=(2500,2500,100) RTT=0.001449209234s
[INFO] [DRONE drone[0]] alerta sintético gerado → drone[0]_1
[INFO] [DRONE drone[0]] VictimAlert drone[0]_1 → team0 (10.0.0.2)
[INFO] [TEAM team0] *** ALERTA de drone[0] msgId=drone[0]_1 vitima em (2500,2500) delay=0.000134368468s
[INFO] [TEAM team0] VictimAck drone[0]_1 → drone origem 10.0.0.1
[INFO] [DRONE drone[0]] VictimAck recebido para drone[0]_1 de team0
```

Leitura linha a linha

1. As duas equipes anunciam sua posição/IP (`TeamUpdate`) em momentos diferentes — efeito do `beaconJitter`.
2. O drone confirma a recepção de cada beacon com `DroneStatus` — por isso aparecem dois RTTs, um por equipe.
3. O drone gera um alerta sintético (`drone[0]_1`) e escolhe `team0` — nesta simulação, a única candidata elegível no momento (equipe mais próxima entre as conhecidas).
4. A equipe recebe, imprime `*** ALERTA ***` (marcador visual de entrega bem-sucedida) e calcula o `delay` — tempo entre `sentAt` do alerta e o instante de recepção. Aqui, 0,13 ms: drone e equipe estão a poucos metros um do outro no cenário de teste.
5. A equipe devolve o `VictimAck` ao IP de origem do drone.
6. O drone recebe o ACK e remove o alerta de `pendingAlerts` — ciclo fechado, sem retransmissão necessária.

7 Métricas gravadas ao final (`finish()`)

Ao fim de cada simulação, cada módulo grava contadores (`scalar`) no arquivo `.sca` — é isso que `analysis/process_results.py` lê para calcular as métricas da dissertação.

7.1 Do lado do drone

Scalar	Tipo	Significado
<code>alertsGenerated</code>	contador	alertas sintéticos criados
<code>alertsSentDirect</code>	contador	<code>VictimAlert</code> enviados direto a uma equipe
<code>alertsSentRelay</code>	contador	<code>VictimAlert</code> enviados em relay broadcast
<code>alertsRelayed</code>	contador	alertas de <i>outro</i> drone recebidos e repassados
<code>alertsAcked</code>	contador	<code>VictimAck</code> recebidos (ciclo completo confirmado)
<code>alertsExpired</code>	contador	descartados após <code>maxRetries</code> sem ACK
<code>totalRetries</code>	contador	soma de tentativas de store-forward
<code>meanRTT</code>	média	tempo alerta→ACK, do lado do drone (ciclo completo, não é o atraso 1-via)

7.2 Do lado da equipe

Scalar	Tipo	Significado
<code>alertsReceived</code>	contador	<code>VictimAlert</code> únicos recebidos (após deduplicação)
<code>teamUpdatesSent</code>	contador	beacons enviados
<code>droneStatusReceived</code>	contador	respostas de drones recebidas
<code>meanDeliveryDelay</code>	média	atraso de entrega 1-via (drone→equipe) — esta é a métrica de “atraso fim-a-fim” reportada na dissertação

PDR vs. AppACK — por que existem duas métricas de sucesso

PDR = `alertsReceived/alertsGenerated` mede *chegada ao destino* (a equipe recebeu). **AppACK** = `alertsAcked/alertsGenerated` mede o *ciclo completo* confirmado (o drone soube que chegou). Com AODV multi-hop os dois quase coincidem, porque o `VictimAck` também consegue voltar vários saltos — a diferença residual vem de ACKs perdidos no caminho de volta.

8 Glossário rápido

Termo	Significado
AODV	Protocolo de roteamento reativo multi-hop — descobre rotas sob demanda (RREQ/RREP), sem tabelas estáticas
RREQ / RREP	<i>Route Request</i> / <i>Route Reply</i> — mensagens do AODV para descobrir e confirmar uma rota
SNR / BER	Relação sinal-ruído / taxa de erro de bit — determinam se um quadro de rádio é decodificado com sucesso
ARP	Protocolo que resolve IP → endereço MAC antes de enviar um quadro no enlace
FANET	Flying Ad-hoc Network — rede ad hoc formada pelos próprios drones em voo
Relay	Repasse de um alerta por um drone que não tem equipe conhecida, esperando que um vizinho tenha
Store-forward	Estratégia de reter o alerta e reenviar periodicamente até receber ACK ou esgotar tentativas
Dedup / <code>seenAlerts</code>	Conjunto de <code>msgId</code> já vistos, evita processar/retransmitir o mesmo alerta duas vezes
Express-mode	Modo do <code>Cmdenv</code> que roda sem imprimir eventos (rápido); precisa ser desligado para ver logs
EV_INFO / EV_WARN	Macros do OMNeT++ para logging em diferentes níveis de severidade
Scalar (<code>.sca</code>)	Arquivo com os contadores finais de cada módulo, um por seed
Seed / run	Uma execução independente da simulação com número aleatório diferente — várias seeds dão média $\pm$ desvio

Gerado a partir do código em `src/app/{SimpleDroneApp,SimpleTeamApp}.{h,cc,ned}` e de uma execução real de `SmokeTest_Beacons` — ver `CLAUDE.md` para detalhes de arquitetura.